



# 1A. OVERVIEW OF DATABASES

# In this lecture – learning objectives

1

- Understand what **databases** are and when/why they are useful.

2

- Be familiar with what a **database management system (DBMS)** is and what functions they tend to provide.

3

- Understand the structure and assessment of this class.

4

- Be familiar with **relational** databases, the most common kind of database.

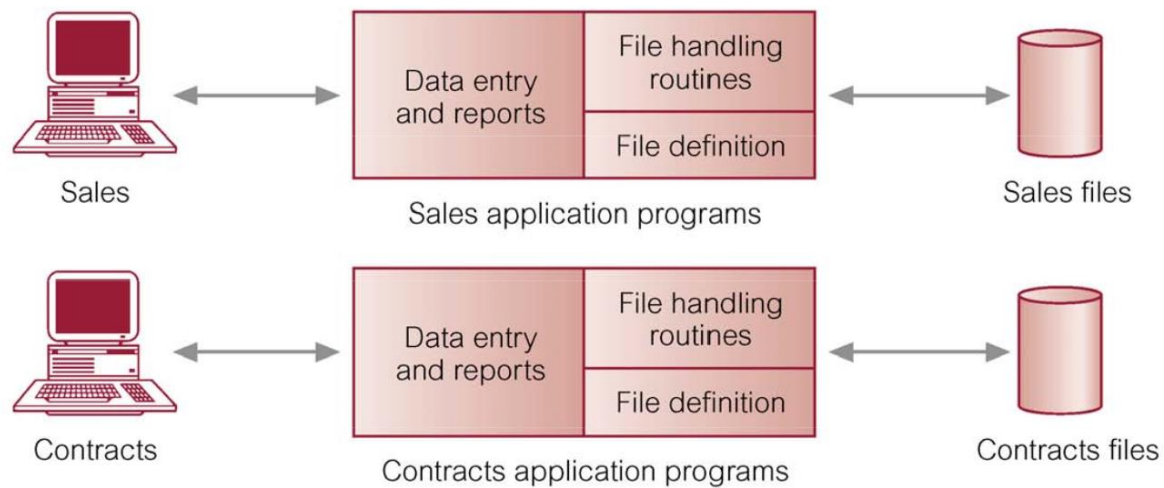
# Importance of data & databases

- Organizations generate data.
  - Employees, documents, products, sales, etc.
- Behind every web application is a database.
  - Social media / web 2.0 site: users, friends/subscriptions, user-generated content (posts, photos, etc.), account settings, usage statistics.
- Important to be able to load, search, and analyze this data.

# Why do we need databases?

- Contrast with older paradigm: file-based systems.
  - Individual applications defining and using their own data types,
  - Unaware of existence of other data,
  - Unintentional duplication of data (inefficient, hard to maintain consistency),
  - Heterogeneous file formats,
  - Analysis and integration of information from multiple sources may be difficult.
- A database **centralizes** data so that it can be stored and managed in one place, standardizing format and minimizing duplication.

# File-based structure



- Each application defines and manages its own data.

## Sales Files

**PropertyForRent** (propertyNo, street, city, postcode, type, rooms, rent, ownerNo)

**PrivateOwner** (ownerNo, fName, lName, address, telNo)

**Client** (clientNo, fName, lName, address, telNo, prefType, maxRent)

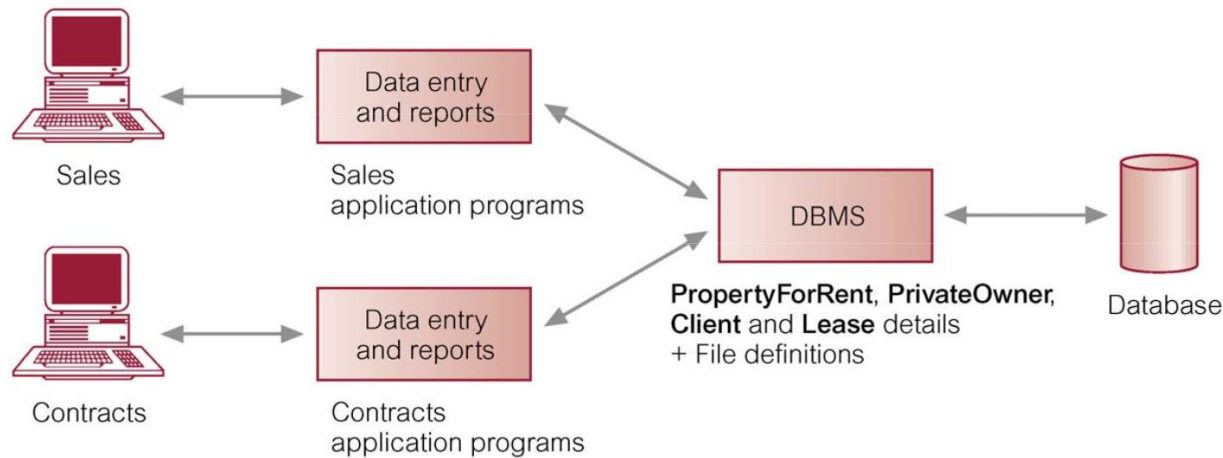
## Contracts Files

**Lease** (leaseNo, propertyNo, clientNo, rent, paymentMethod, deposit, paid, rentStart, rentFinish, duration)

**PropertyForRent** (propertyNo, street, city, postcode, rent)

**Client** (clientNo, fName, lName, address, telNo)

# Centralized database



**PropertyForRent** (propertyNo, street, city, postcode, type, rooms, rent, ownerNo)

**PrivateOwner** (ownerNo, fName, lName, address, telNo)

**Client** (clientNo, fName, lName, address, telNo, prefType, maxRent)

**Lease** (leaseNo, propertyNo, clientNo, paymentMethod, deposit, paid, rentStart, rentFinish)

- Multiple applications share a database managed by software.

# A database is...

- “a shared collection of logically related data (and a description of this data), designed to meet the information needs of an organization”
- “a *self-describing* collection of integrated records”
  - This *description* of a database’s format is called a **schema**.

# A database management system is...

- “a software system that enables users to define, create, and maintain a database and that provides controlled access to this database”

# Functions of a database management system (DBMS)

- Database management systems (DBMSs) also centralize **useful functions** for managing and accessing that data across an organization/project :
  - Methods of data access,
  - Performance optimizations,
  - Security,
  - Ensuring data integrity,
  - Update failure recovery.
- Downsides:
  - Increased size / complexity / cost.

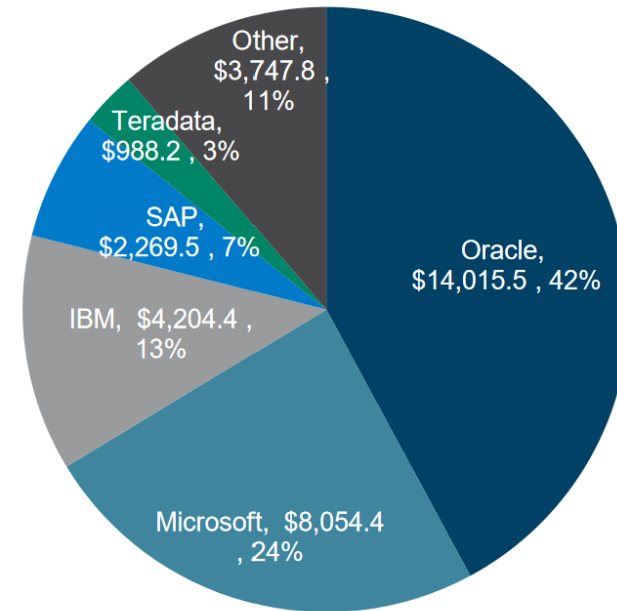
# Practical considerations when using DBMS software

- Different **types** of databases exist.
  - Different ways of handling data and data storage, each with their own pros/cons/situations when useful.
- Need a way to set up and interact with a database.
  - **Data definition language (DDL)** and **data manipulation language (DML)** (more on these later).
- Multiple DBMS products (software) exist:
  - Various software companies have implemented DBMSs suitable for managing different types of databases.
  - Some DBMS software is free, open-source, paid, or proprietary.

# Database software

- Relational database software:
  - Oracle
  - Microsoft Access / SQL Server
  - IBM Db2
  - SAP
- Open-source:
  - MySQL
  - MariaDB
  - PostgreSQL
  - MongoDB

Relational Database Vendor Share 2017 (in millions) (\$33.1B in total)



\*Data includes analytical databases

Sources: Worldwide Relational Database Management Systems Software Market Shares, 2017: The Race to the Cloud, IDC, Published June 2018

From: "Database Software Market: The Long-Awaited Shake-up," William Blair, March 2019

# Main types of databases

- *Relational* database management systems (RDBMS): the most mature and still dominant type of database.
  - The overwhelming focus of this module.
- Before relational databases: hierarchical databases.
- Current alternatives: document-oriented (especially using JSON).

# Rest of this module

- First two weeks:
  - How to use relational DBs? – SQL; designing relational databases.
- Weeks 3-6:
  - How are relational DBs implemented?
    - Relational algebra, storage, query processing & optimization, transaction management & recovery.
- Weeks 7-8:
  - Distributed databases.
- Week 9-10:
  - Beyond RDBMSs.

# Early weeks

- Coursework (50%) involves designing and making use of your own SQL database.
  - Material needed to get started on this task is front-loaded into the first two weeks.
- Week 1:
  - Introduce yourself / form groups (on Moodle).
  - Understand SQL (exercises on Moodle).
- Week 2:
  - Learn about ER diagrams and normalization.
  - Finalize groups and peer-marking criteria by end of Week 2.
- Week 3:
  - Submit an ER diagram for your project by end of Week 3.

# Rest of this lecture

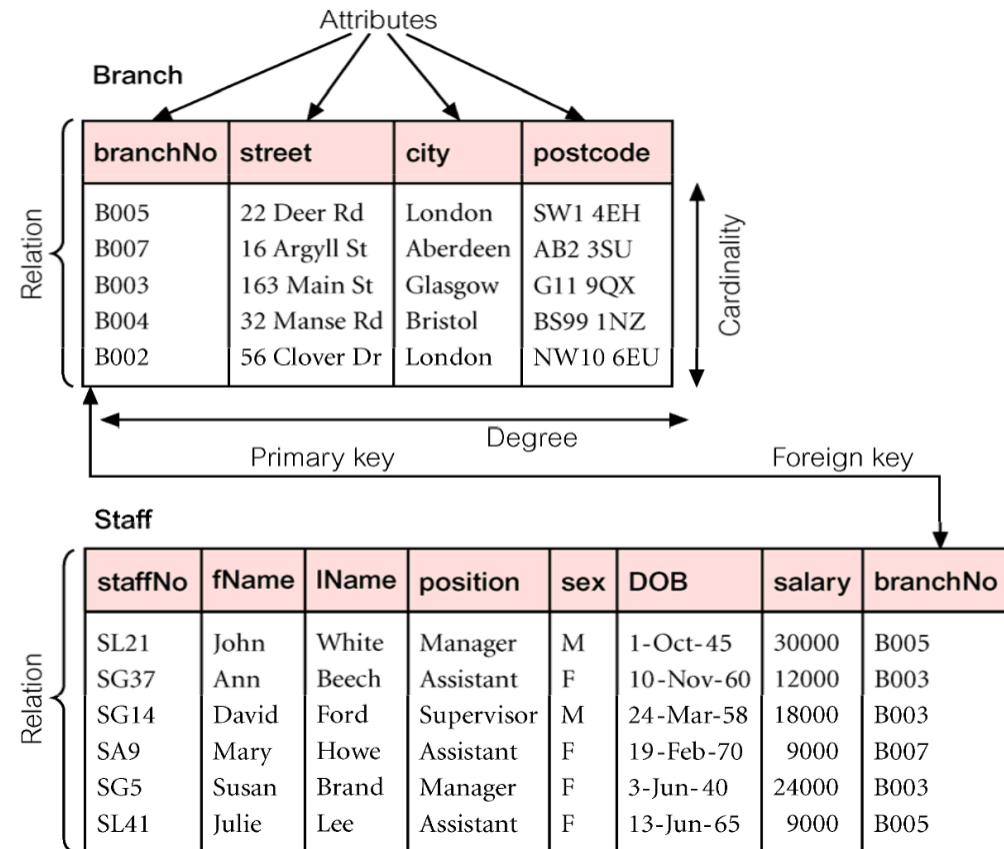
- Introduction to databases.
- Introduction to relational databases.
- Generic architecture of database management systems.



# Intro to relational databases

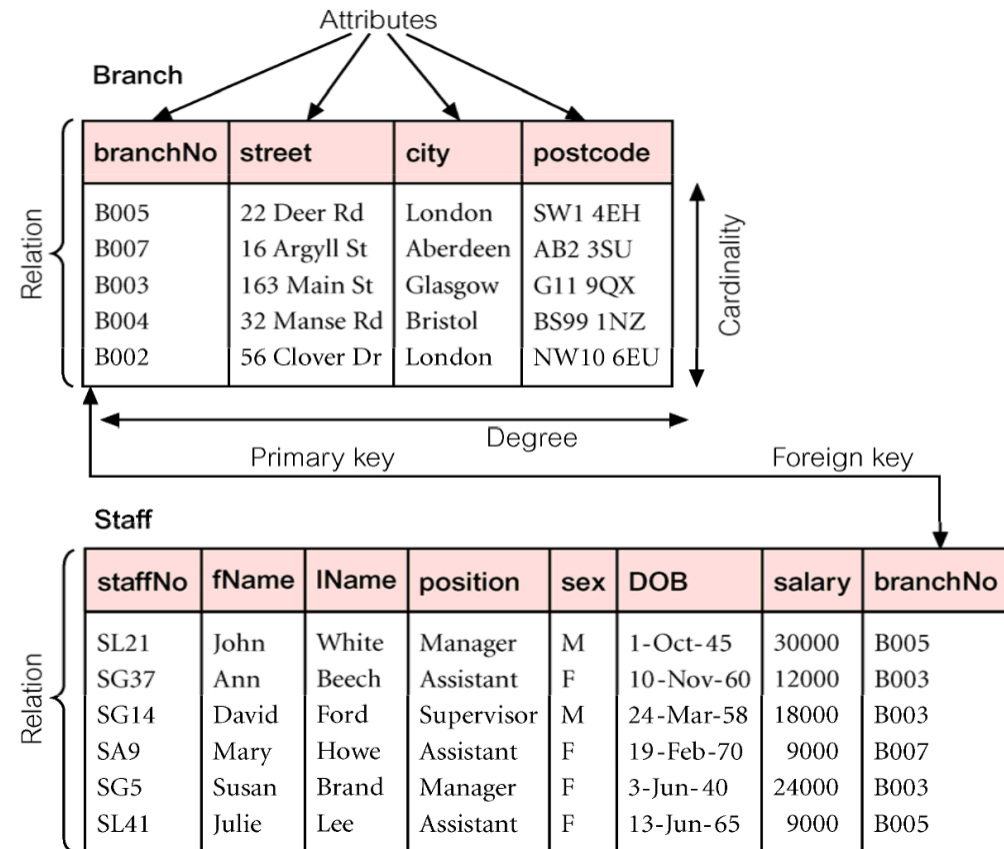
# Tables

- For now, can think of the basic unit of a relational database as being a 2-D table.
  - We'll go over this in more detail later.
- One table represents a thing, entity, relationship (Capitalized)
  - Branch; Staff
- Columns = "attributes" or fields (lowerCamelCase)
  - Domain: set of allowable values
- Rows = "records"



# Terminology & key concepts

- Degree = # cols
- Cardinality = # rows
- Every table has a primary key (underlined), an attribute(s) that uniquely determines a record.
  - Branch ( branchNo, street, city, postcode )



# Database = set of tables

- Need to define domains of attributes in all tables (data definition).
- Need to populate tables with records (data manipulation).

# Simple structure!

- Similar to an Excel table.
- Regular (rectangular) structure.
- Easy to analyze and manipulate.

# Relational database technologies

- Relational database management systems (RDBMS) – a mature technology that is...
  - **Formalized** by relational algebra/calculus.
  - **Standardized** by the SQL (Structured Query Language) standard.
  - **Implemented** by various off-the-shelf products (MySQL, Oracle, MariaDB, SQLite, PostgreSQL Microsoft SQL Server, IBM Db2, etc.).



# Design & architecture of a DBMS

# Examining the DBMS

- Recall: DBMS software collects functions useful for operating a centralized database.
- Topics:
  - Communicating with DBMSs as a user.
  - Database descriptions at different "layers."
  - Subparts (functions, modules) of a DBMS.
  - How DBMSs support multiple users.
    - Databases and the web.

# Database definition and querying languages

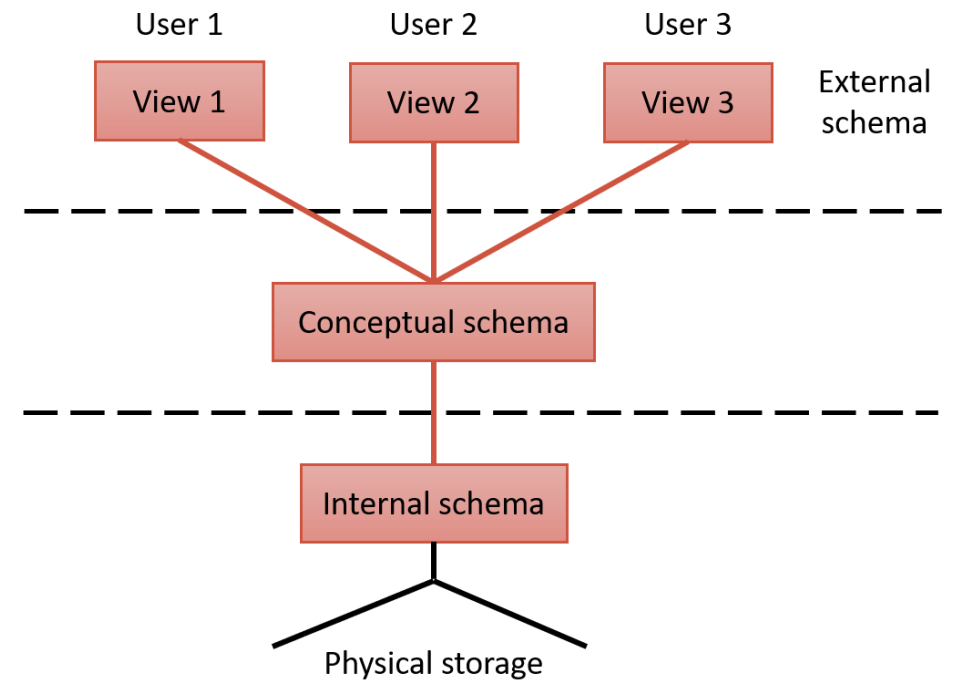
- **Data definition language (DDL)** – specifies format of data in the database (**database model** or **database schema**).
  - Includes constraints on data (integrity checks) and permissions/security.
- **Data manipulation language (DML)** – standardized way of specifying updates to data in the database.
- => For relational databases, SQL is both a DDL and DML.

# Helpful definitions

- Schema: A description of an organization's data.
- Conceptual (schema): A description of data agnostic of type of database (relational, etc.)
  - We'll talk about this more next week – database design and ER diagrams
- Logical (schema): Description of database relative to a specific database type.
  - In relational DBs, this is the full set of tables and other metadata.
- Physical (storage): How data is stored on a specific machine.

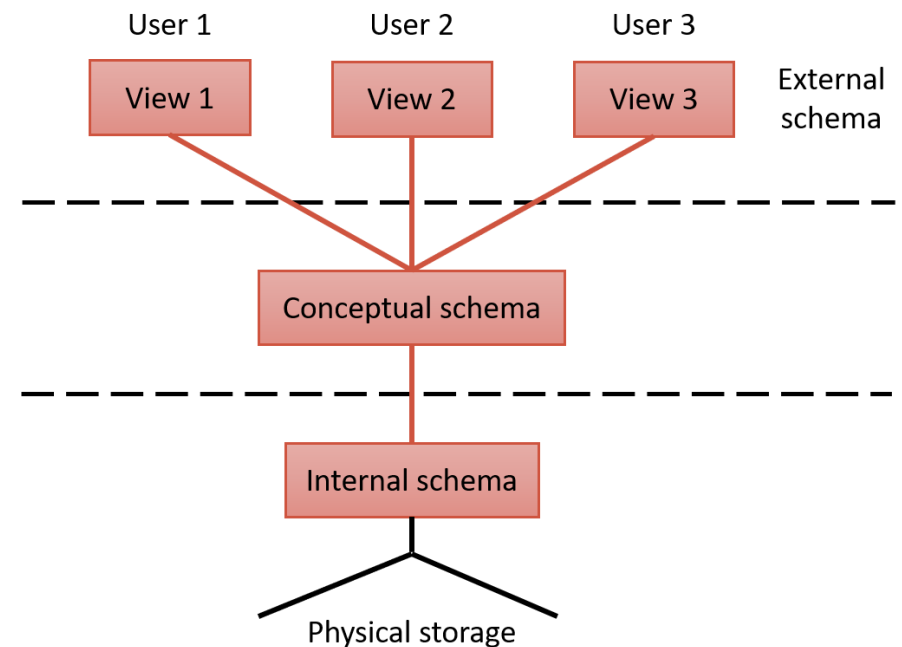
# Three-tier DB architecture

- External level – views and subschemas:
  - Views of the data accessible by users.
  - Can be set per user and hide details they don't need to / shouldn't know.
- Conceptual level:
  - Full specification of the logical/conceptual structure of the database, minus platform-dependent implementation details.
- Internal / physical level:
  - Data structures and algorithms defining how data is physically stored on disk.



# Abstraction & interfaces

- Each layer abstracts away details from previous layers.
- Allows changes in one layer with minimal impact at other layers (reduced coupling).
- Flexibility of specialized applications (decentralized) + unified, comprehensive storage/management of data (centralized).

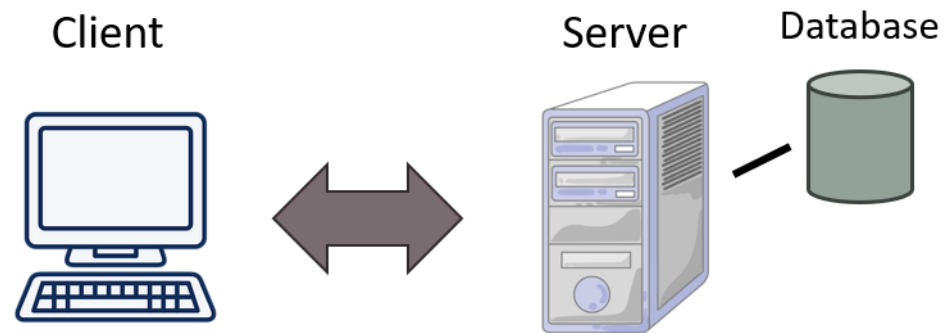


# Common parts of a DBMS

- Query processor: translates queries into low-level instructions.
- Database manager: large and complicated piece of code with many functions such as authorization, integrity-checking, query optimization, scheduling, failure recovery, and temporary data storage.
- File manager: stores data and auxiliary data structures on physical hardware.
  - Interacts with operating system as necessary.

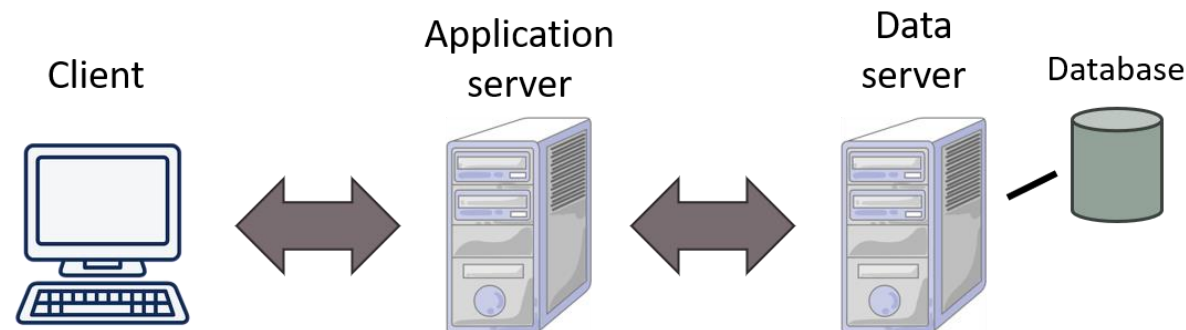
# Multi-user databases

- How to set up a single database so that it can be used by multiple users / applications?
- Has changed over the years and settled into **client-server** architecture.
  - Client (frontend): [example: web browser] makes requests, handles interactivity, application logic maybe involving light data-processing.
  - Server (backend): handles DB queries + storage.



# Three-tier client-server model

- Server can be further split into two parts: data server and application server.
- Three tiers:
  - Thin client: display + UI (web browser).
  - Application server: all application logic, request handling.
  - Data server: data storage + retrieval.
    - Possible to further split data server in high-traffic DBs into a transaction manager (queues/schedules updates and handles rollbacks) and resource manager (read/write from file).



# Cloud computing

- **Cloud:** “A model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g. networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction.”
- On-demand / self-service.

# In-house vs. cloud-managed databases

- Traditional model: locally-managed data storage on own computers/computer system, installed with DBMS software.
- Trend toward: Using **database as a service (DBaaS)** – a type of **software as a service (SaaS)** where a company provides database hosting.
  - Remote user configures database but does not handle storage of it.

# DBaaS vendors

- Examples:
  - Amazon AWS,
  - Microsoft Azure,
  - Google Cloud,
  - Cloud versions of IBM Db2, SAP, etc.
- Often provide multiple options (relational, non-relational, etc.).

# Summary

- Organizations make use of data.
- Useful data can be centralized into a **database**.
- Useful database functions can be centralized into a **database management system** (DBMS).
- Relational databases are a simple, mature, easily-analyzed type of database.
- Next time: *SQL* – the standard data definition/manipulation language for relational (and other) databases.